Torch

torch.index_select()
torch.tensor.detach().cpu().numpy()

- device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

gradient clipping simple solution to gradient explosion, works well practically

- torch.nn.utils.clip*grad_norm*(parameters, max_norm, norm_type=2)
  - example: torch.nn.utils.clip*grad_norm*(retinanet.parameters(), 0.1)

Tensor shapes

From Simple BEV repo: We maintain consistent axis ordering across all tensors. In general, the ordering is `B, S, C, Z, Y, X`, where

- B: batch
- S: Sequence (for temporal or multiview data)
- C: channels
- Z: depth
- Y: height
- X: width
  The ordering stands even if a tensor is missing some dims, for example, plain images are `B, C, Y, X` (as is the pytorch standard).

Axis directions

- Z: forward
- Y: down
- X: right
  This means the top-left of an image is "0.0", and coordinates increase as you travel right and down. `Z` increases forward because it's the depth axis.

1. Chapter 2
   1. ```
      import torch
      device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

      net.to(device)
      images = images.to(device)
      labels = labels.to(device)
      output = net(images)
      loss = criterion(output, labels)
      ```

   ### Pytorch
   #### Tensor
   1. From an interface perspective, operations on Tensors can be divided into two categories:
      1. `torch.function`, such as `torch.save`, etc.
      2. `tensor.function`, such as `tensor.view`, etc.
   2. For ease of use, most operations on Tensors support both types of interfaces simultaneously, and this book does not make a specific distinction, for example, `toch.sum(a, b)` and `a.sum(b)` are functionally equivalent.
   3. From a storage perspective, operations on Tensors can be divided into two categories:
      1. A function that does not modify its own data, such as `a.add(b)` will return a new Tensor as the result of the addition.
      2. A function that modifies its own data, such as `a.add_(b)` will still store the result if the addition in `a`, but `a` will be modified.
      ```

4. Functions whose names end with an underscore are inplace functions, meaning they modify the caller's own data. This distinction is important in practical applications.

Creating Tensors

1. There are many ways to create tensors in Pytorch, as shown in Table 3-1.

Function	Functionality
Tensor(*sizes)	Basic constructor
tensor(data,)	Constructor similar to np.array()
ones(*sizes)	Tensor with all ones
zeros(*sizes)	Tensor with all zeros
eye(*sizes)	The diagonal is 1, and the rest are 0
arange(s, e, step)	From s to e, the step size is step
linspace(s, e, steps)	From s to e, divide evenly into steps
rand/randn(*sizes)	uniform/standard distribution
normal(means, std)/uniform(from, to)	normal distribution/ uniform distribution
randperm(m)	random arrangement
tensor.new_*/torch.*_like	create a Tensor of the same size, filled with * type (e.g. tensor_new_ones, is filled with all ones), and with the same torch.dtype and torch.device

2. All these creation methods allow you to specify the data type (dtype) and the storage device (CPU/GPU) during creation.
3. The tensor function can be used to create a new Tensor. It can accept a list and create a new tensor based on the data in the list. It can also create a tensor based on the data in the list. It can also create a tensor based on a specified data shape and can pass in other tensors. Several examples will be given below.
4. It's important to note that when `t.Tensor(\*sizes)` creates a Tensor, the system does'nt immediately allocate space. It only calculates whether there's enough remaining memory and allocates space immediately after tensor creation. In practice we recommend using `torch.tensor` instead of `torch.Tensor`. Let's compare the differences between the two.
5. `torch.Tensor` is a python class, and by default it is `torch.FloatTensor()`. Running `torch.Tensor([2, 3])` will directly call the `__init__()` constructor of the `Tensor` class, generating a single-precision floating-point Tensor(FloatTensor).
6. `torch.tensor` is a python function with the prototype `torch.tensor(data, dtype=None, device=None, requires_grad=False)`. `data` supports types such as list, tuple, array, and scalar. `torch.tensor()` directly copies data from `data` and generates the corresponding Tensor based on the original data type.
7. Since `torch.tensor()` can generate a corresponding Tensor based on the data type, we recommend using the `torch.tensor()` method to create a new Tensor in practical applications.

Types of Tensor

8. Tensor types can be further divided into device type and data type (dtype). Device type is divided into CUDA and CPU types, and numeric types include bool, float, and int.
9. Tensor data types are shown in Table 3-2, and each data type has CPU and GPU versions. Readers can obtain the device type of a tensor using `tensor.device` and the data type using `tensor.dtype`.
10. The default data type of a Tensor is `FloatTensor`. Readers can modify the default Tensor type using `torch.set_default_tensor_type` (if the default type is a GPPU Tensor, then all operations will be performed on the GPU) or `torch.set_default_dtype` (only accepts float input types).
11. Understanding the type of a Tensor is helpful for analyzeing memory usage. For example, a FloatTensor of shape $(1000, 1000, 1000)$ has $1000x1000x1000 = 10^9$ elements, each occupy $32\text{bit} \div 8 = 4\text{byte}$ of memory/GPU memory. HalfTensor is specifically designed for GPU versions. With the same number of elements, HalfTensor uses only halft the GPU memory of a FloatTensor.
12. Therefore, using HalfTensor can greatly alleviate the problem of insufficient GPU memory. It should be noted that HalfTensor has limited numerical values and precision, which may lead to data overflow issues.
13. Tensor data type

Data Type	dtype	CPU tensor	GPU tensor
32-bit floating-point	torch.float32 or torch.float	torch.FloatTensor	torch.cuda.FloatTensor
64-bit floating-point	torch.float64 or torch.double	torch.DoubleTensor	torch.cuda.DoubleTensor
16-bit half-precision floating-point	torch.float16 or torch.half	torch.HalfTensor	torch.cuda.HalfTensor
8-bit unsigned integer	torch.uint8	torch.ByteTensor	torch.cuda.ByteTensor
8-bit signed integer	torch.int8	torch.CharTensor	torch.cuda.CharTensor
16-bit signed integer	torch.int16 or torch.short	torch.ShortTensor	torch.cuda.ShortTensor
32-bit signed integer	torch.int32 or torch.int	torch.IntTensor	torch.cuda.IntTensor
64-bit signed integer	torch.int64 or torch.long	torch.LongTensor	torch.cuda.LongTensor
Boolean type	torch.bool	torch.BoolTensor	torch.cuda.BoolTensor

7. Common methods for converting between different types of tensors are as follows:
   1. The most common approach is `tensor.type(new_type)`, but there are also shortcut methods such as `tensor.float()`, `torch.long()`, and `tensor.half()`.
   2. Conversion between CPU tensors and GPU tensors is achieved using `tensor.cuda()` and `tensor.cpu()`, and `tensor.to(device)` can also be used.
   3. Creating tensors of the same type: `torch.*_like` and `torch.new_*`. These two methods are suitable for writing device compatible code:
      1. `torch.*_like(tensorA)` generates a new tensor with the same attributes as `tensorA` (such as type, shape and CPU/GPU)
      2. `tensor.new_*(new_shape)` creates a new tensor with a different shape but the same attributes.

References:

1. Pytorch book
2. Modern Computer Vision with PyTorch: Explore deep learning concepts and implement over 50 real-world image applications, Kishore Ayyadevara.